

R Programming

Suprit Saha
Tutorial Four

March 15, 2015

1 Lists

A list is a collection of objects. The elements of a list can be different types of object and can be of different lengths. Plays a central role in R and forms a basis for data frames & object oriented programming.

- Creating Lists

```
1 x<-list( first=1:10, second=c("yes", "no"), third=c(T,F), fourth= gl(2,3) )
2 x1<-list(1:10, c("yes", "no"), c(T,F), gl(2,3))
3 #Alternative
4 y<-vector(mode="list")
5 y[["first"]]<-1:10
6 y[["second"]]<-c("yes","no")
7 y[["third"]]<-c(T,F)
8 y[["fourth"]]<-gl(2,3)
```

Note: Using names in lists makes it clearer & less error prone. Lists are also vectors so they can be created via vector().

- Indexing

```
1 x$first #Extract component of list
2 x[[1]] # Alternative
3 x[[first]] # Alternative
4 x[1:3] #Extract more than 1 component
5 x[[first]][5] #Extract a particular element of component
```

Note: Use [[]] to access list components

- Adding/Deleting List Elements

```

1 z <-list(a="abc", b=12)
2 z$c <- "New" #Adding
3 z[[4]] <-27
4 z[5:7] <-c(FALSE,TRUE,FALSE)
5 z$b <-NULL #Deleting
6 z1<-c( z, list(last=matrix(1:6,3,2))) #Concatenate lists

```

- unlist() & names() function

```

1 l<-list(a=1:100,b=2:5)
2 names(l) #Get tags of list
3 unlist(l) # List to vector
4 list1<-list(name="Joe", salary=5500,Graduate=TRUE)
5 unlist(list1)
6 names(list1)<-NULL #Remove names from list
7 #Alternative
8 list1<-unname(list1)

```

Note: unlist() function flattens the list into a vector, and coerce types so that all elements are of the same type.

- lapply() & sapply() functions

Two functions are handy for applying functions to lists: **lapply** and **sapply**

```

1 x<-list(a=1:5,b=rnorm(100), c=c(23,45,67))
2 lapply(x, median)
3 lapply(x, function(y) (max(y)+min(y))/3)
4 lapply(x, function(y) c(min(y),max(y),mean(y),median(y)))
5
6 sapply(x, median)
7 sapply(x, function(y) (max(y)+min(y))/3)
8 desc<-sapply(x, function(y) c(min(y),max(y),mean(y),median(y)))
9 rownames(desc)<-c("Min", "Max", "Mean", "Median")

```

Note: lapply() loops over a list and evaluates a function on each component. sapply() is a user-friendly version of lapply() by default returning a vector or matrix if appropriate.

lapply() is the workhorse of many other ***apply** functions. Peel back their code and you will find lapply underneath.

2 Data Frames

A data frame is like a matrix, with a two dimensional row & column structure. However each column may have different modes. Technically, a data frame is a list of equal length vectors. Most of the real life datasets are data frames.

- **Creating data frames**

```
1 kids=c("Jack", "Jill")
2 ages<-c(12,10)
3 d<-data.frame( kids, ages, stringsAsFactors=FALSE)
```

Note: By default all character vectors are converted to factors while creating data frames

- **Accessing data frames**

```
1 #Since data frame is a list
2 d[[1]] #First column
3 d$kids
4 #Since data frame is matrix like
5 d[, 1] # First column
6 d[, colnames(d)=="kids"]
7 d[, "kids"]
```

Note: It is clearer & safer to extract columns by column names

- **Filtering**

```
1 data<-read.table("C:\\Users\\Suprit\\Desktop\\example.txt",header=TRUE)
2 data[2:5, ] # 2nd to 5th rows
3 data[2:5, 4] # 4th column of above
4 data[2:5, 4, drop=FALSE] # Retains the data frame structure
5 data[data$maths >80 & data$statistics >90, ]
6 #Alternative
7 subset(data, maths >80 & statistics >90)
```

Note: Mostly imported data sets are stored as data frames

- **rbind() & cbind()**

```
1 kids=c("Jack", "Jill")
2 ages<-c(12,10)
3 d<-data.frame( kids, ages, stringsAsFactors=FALSE)
4 d<-rbind(d, list("Laura ",19))
5 d<-cbind(d, c(89,45,76))
6 names(d)[3]<- "Marks"
```

Note: rbind() & cbind() are used to append data frames

- **apply, lapply & sapply**

```
1 data<-read.table("C:\\Users\\Suprit\\Desktop\\example.txt",header=TRUE)
2 apply(data[,3:5],1,max)
3 data1<-lapply(data, sort)
4 data2<-sapply(data[,3:5], function(x) c(sum(x),mean(x)))
5 rownames(data2)<-c("Total Marks", "Average Marks")
```

Note: Since data frame is both a list & matrix like we are able to apply all three functions

- **attach(), detach() & with()**

The attach() function adds the data frame to the R search path. The detach() function removes the data frame from the search path

```
1 attach(data)
2 statistics
3 maths + computer
4 detach(data)
5 #Alternative
6 with(data, {
7   print(statistics)
8   print(maths + computer)})
```

Note: The attach()function in R makes objects within data frames accessible in R with fewer keystrokes. Should be avoided since there may be multiple objects with same name.

3 Data Management

Getting our data in the rectangular arrangement of a matrix or data frame is the first step in preparing it for analysis. Before starting to apply statistical methods on our dataset,generally some data cleaning/tidying steps are required.

Let us consider the following dataset:

Manager.ID	Country	Gender	Age	q1	q2	q3	q4	q5
2311	US	M	24	5	4	5	5	5
4066	US	F	26	3	5	2	5	5
2522	UK	F	45	3	5	5	5	2
3073	UK	M	34	3	3	4		
2198	UK	F	99	2	2	1	2	1
2522	UK	F	45	3	5	5	5	2

- **Creating the dataset**

```
1 Manager.ID <- c(2311, 4066, 2522, 3073, 2198, 2522)
2 Country <- c("US", "US", "UK", "UK", "UK", "UK")
3 Gender <- c("M", "F", "F", "M", "F", "F")
4 Age <- c(32, 45, 25, 39, 99, 45)
5 q1 <- c(5, 3, 3, 3, 2, 3)
6 q2 <- c(4, 5, 5, 3, 2, 5)
7 q3 <- c(5, 2, 5, 4, 1, 5)
8 q4 <- c(5, 5, 5, NA, 2, 5)
9 q5 <- c(5, 5, 2, NA, 1, 2)
10 data <- data.frame(Manager.ID, Country, Gender, Age,
11 q1, q2, q3, q4, q5, stringsAsFactors=FALSE)
```

Note: Here all string variables are considered to be **character** variables rather than **factor** which is the default type.

- **Structure of the dataset**

```
1 dim(data) #Dataset dimensions
2 str(data)
```

Note: The str function compactly illustrates the internal structure of any R object

- **Type Conversions**

```
1 data$Manager.ID <- as.character(data$Manager.ID)
```

Note: Always perform the necessary type conversions

- **Removing duplicates**

A very useful application of subsetting data is to find and remove duplicate values. R has a useful function, **duplicated()**, that finds duplicate values and returns a logical vector that tells you whether the specific value is a duplicate of a previous value. This means that for duplicated values, duplicated() returns FALSE for the first occurrence and TRUE for every following occurrence of that value

```
1 data <- data[!duplicated(data),]
2 #Alternative
3 data <- unique(data)
```

- **Adding new columns**

```
1 data$Income <- c(100,100,150,150,50)
```

Figure 1: Type Conversion Functions

Test	Convert
<code>is.numeric()</code>	<code>as.numeric()</code>
<code>is.character()</code>	<code>as.character()</code>
<code>is.vector()</code>	<code>as.vector()</code>
<code>is.matrix()</code>	<code>as.matrix()</code>
<code>is.data.frame()</code>	<code>as.data.frame()</code>
<code>is.factor()</code>	<code>as.factor()</code>
<code>is.logical()</code>	<code>as.logical()</code>

- **Rename variables**

```

1 # Get column headers
2 names(data)
3
4 # Editing column names
5 names(data)[5:9] <- c("Item1", "Item2", "Item3", "Item4", "Item5")

```

- **Recoding variables**

Recoding involves creating new values of a variable conditional on the existing values of the same and/or other variables. For example, you may want to

- Change a continuous variable into a set of categories
- Create a pass/fail variable based on a set of cutoff scores
- Replace miscoded values with correct values

To recode data, you can use one or more of R's logical operators

```

1 data$Age[data$Age == 99] <- NA
2 data$Age.category[data$Age < 30] <- "Junior"
3 data$Age.category[data$Age >= 30 & data$Age < 40] <- "Midlevel"
4 data$Age.category[data$Age >= 40] <- "Senior"
5 data$Age.category <- factor(data$Age.category, levels=
6 c("Junior", "Midlevel", "Senior"), ordered=TRUE)

```

- Missing observations

```
1 #Delete missing observations
2 newdata<-na.omit(data)
```

Note: You can remove any observation with missing data by using the `na.omit()` function. `na.omit()` deletes any rows with missing data.

- Sorting

```
1 data[order(data$Age), ] # in increasing order of age
2 data[order(data$Age, decreasing=TRUE), ]
3 data[order(data$Age, data$Income), ] # more than 1 variable
```

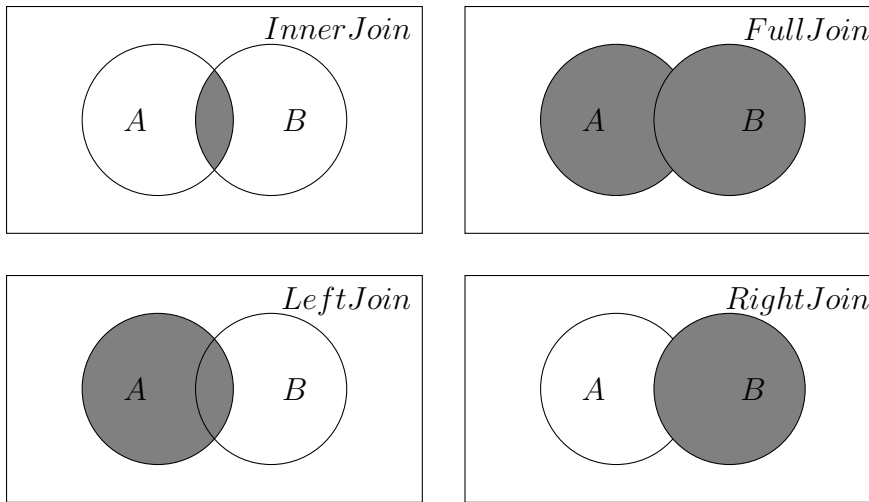
- Subsetting & Dropping variables

```
1 #Subsetting data
2 data[,5:9]
3 data[,-(5:9)]
4
5 #Dropping variables
6 data$Age.category <- NULL
```

- Selecting Observations

```
1 data[data$Gender == "M", ]
2 data[data$Gender == "M" & data$Age > 30, ]
3 data[data$Age > 30, ] # NA values are included
4 subset(data, data$Age>30) # useful when NA values are present
5 subset(data, data$Age>30, select=c("Country","Income"))
```

- Merging



```
1 #Merging
2 data2 <- read.csv("C:\\Users\\Suprit\\Desktop\\RFiles\\
3 5.Tutorial Five\\leadership data2.csv", header=TRUE)
4 #Inner join
5 merge(data, data2, by="Manager.ID")
6 #Full join
7 merge(data, data2, by="Manager.ID", all=TRUE)
8 #Left join
9 merge(data, data2, by="Manager.ID", all.x=TRUE)
10 #Right join
11 merge(data, data2, by="Manager.ID", all.y=TRUE)
```